UNIT-2

MACHINE LEARNING

1. Linearity Vs non Linearity
2. Activation functions like sigmoid, ReLU.
3. Gradient Descent
4. Multilayer Network
5. Back Propagation.
6. Auto encoders
7. Batch Normalization.
8. L1 and L2 Regularization.
9. Tuning Hyper parameters.

Important Topics

PGRVALL CONTENT

UNIT-2

Machine Learning.

Linearity Vs non linearity

Linearity A model is linear when the relationship between input and output is a straight line (or a linear combination of inputs)

Mathematical form -

$$y = w_1 x_1 + w_2 x_2 + \ldots + b$$

Main Points.

Output changes proportionally with input

It is easy to compute and interpret.

Decision boundary = straight line (2D) or plane (3D).

Example →

- Predicting salary based on experience.
- linear regression.

RGPV ALL CONTENT

Non-Linearity

A model is non-linear when the relationship is curved or complex, not a straight line.

Example → $y = x^2$
or
$y = \sin(x)$

Main Points.

- It captures complex patterns
- Decision boundary - curve or irregular shape.
- Needed for real-world problems like images, speech.

Importance of Non-Linearity.

If we only use linear models:-

Even multiples layers behaves like a single linear model

suppose → $y = w_2(w_1 x)$

This is still linear!

So, without non-linearity, deep learning is useless.

RGPV ALL CONTENT

Activation function.

Neuron → A neuron is the smallest unit of a neural network that.

Take input.
Processes it
Produces an output

It is inspired by human brain neuron

Activation function - An activation function is a mathematical function applied to neuron's output :-

$$y = f(Wx + b)$$

Y = Activation (Σ (weight * input) + bias).

Activation function decides, whether neuron should activated or not by calculating weighted sum and adding bias with it to introduce non-linearity in the output of neuron.

It used to determine the output of neural network like yes or no, It maps the resulting values in between 0 to 1 or -1 to 1 etc. (depending upon the function).

RGPV ALL CONTENT

Activation function are an extremely important feature of the artificial neural networks.

Without Activation Function

- Neural network becomes linear.
- Cannot solve complex problems.

With Activation function

Introduces non-linearity.

And Enables:

- Image recognition
- Speech processing
- NLP tasks.

Important Types of Activation function

1. Sigmoid Activation function
2. ReLU (Rectified Linear Unit)

RGPV NOTES CONTENT

① Sigmoid.

The sigmoid activation function is a mathematical function that converts any real value into a value between 0 and 1.

It is also called the logistic function

It is easy to work with and has all the nice properties of activation functions, it's non-linear, continuously differentiable, monotonic, and has a fixed output range.

Mathematical formula.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

where

- x = input to the neuron
- e = Euler's constant

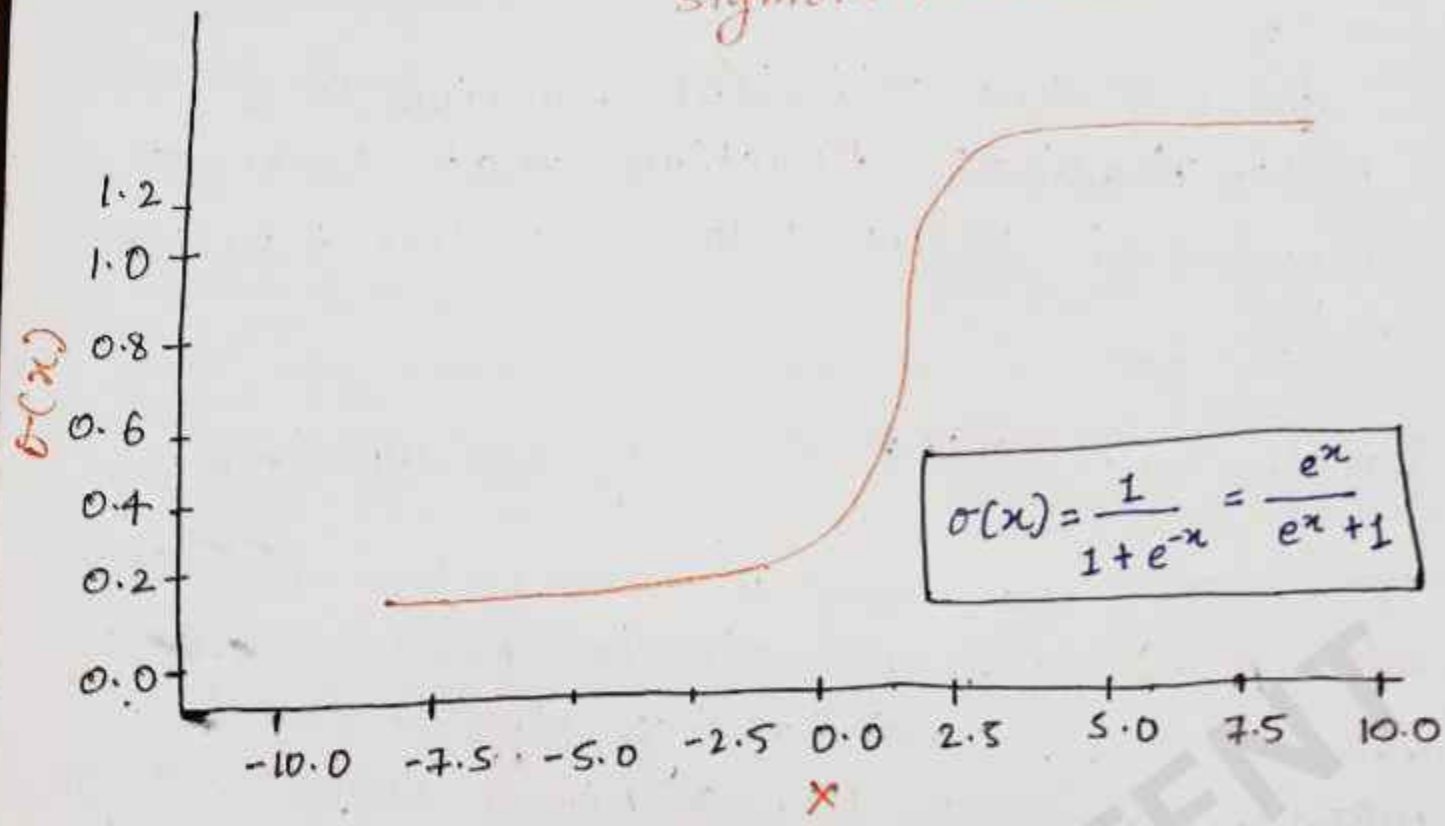
Output range →

$$0 < \sigma(x) < 1$$

- Output is always between 0 and 1
- Never exactly 0 or 1

RGPV ALL CONTENT

Sigmoid function



$$\sigma(x) = \frac{1}{1+e^{-x}} = \frac{e^x}{e^x+1}$$

Derivative

$$\sigma'(x) = \sigma(x)(1-\sigma(x))$$

Easy to compute
Used in backpropagation.

Advantages / Pros

1→ Produces values between 0 & 1, so it can be interpreted as probability. and useful in binary classification.

2- The function is continuous and smooth.

PGV ALL CONTENT

3. Good in output layer especially in Yes / no problems and True / False predictions.

4. It is Monotonic function means always increasing.
   - Higher Input → higher output.
   - no sudden jumps

Disadvantages / Cons.

(1) Vanishing Gradient Problem occurs means for large positive or negative values.
Gradient becomes very small (≃0).

(2) A extreme values:
   - Output becomes almost constant (0 or 1)
   - because of this model stops learning.

RGPV ALL CONTENT

(2) ReLU

- ReLU (Rectified Linear Unit) is a non-linear activation function
- It provides the same benefits as sigmoid but with better performance.
- It transforms input as:

$$f(x) = \max(0, x)$$

- If input is negative → output is 0.
- If input is positive output is same value

Importance.

→ Without activation

- Neural network becomes linear.

→ With ReLU.

- It introduces non-linearity.
- Helps model learn complex patterns.

RGHV ALL CONTENT

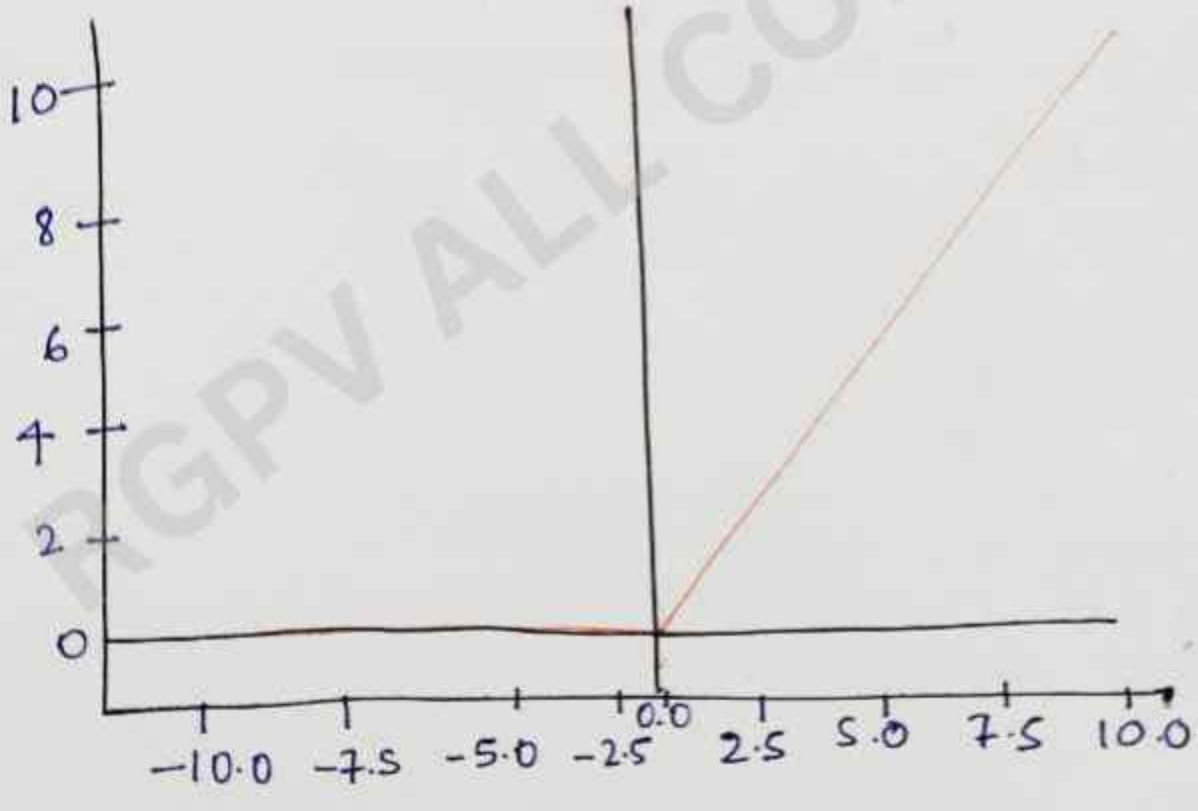
$$f(x) = \max(0, x).$$

where

- x is the input to the neuron.
- The function returns x if x is greater than 0.
- If x is less than or equal to 0, the function return 0.

The formula can also written as:

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$



ReLU Activation function

RGPV ALL CONTENT

Parametric ReLU (PReLU)

→ Learns slope automatically for negative values.

ELU (Exponential Linear Unit.

→ Smooth version of ReLU.

where is ReLU used

→ Mostly in

- Hidden layers of neural networks
- Deep learning models.

RGPV ALL CONTENT

What are Weights and Bias in Neural Networks.

A neuron (like a small calculator) works like this :-

$$Output = (w \cdot x) + b$$

where.

x : input
w = weight
b = bias.

Weight

Weight is a numerical parameter in a neural network that determines the strength and importance of an input feature in producing the output.

weight tells - How much influence does the input have on the final result?

Representation

for single input neuron :-

$$y = w \cdot x$$

+ for multiple inputs.

$$y = w_1x_1 + w_2x_2 + w_3x_3 \ldots\ldots w_nx_n$$

PRIVATE CONTENT

Where.

x_1, x_2, x_3 = inputs

w_1, w_2, w_3 = corresponding weights.

Each input has its own ~~input~~ weight.

Role of Weight

Weights play a very important role in neural network.

Feature Importance

High weight - input is very important

Low weight - input is less important.

Decision Making

- Output of neuron depends mainly on weights
- They help the model decide correct predictions.

Pattern learning

- Neural network learns pattern by adjusting weights.

TOPPVALL CONTENT

Real life example

→ Suppose we are predicting student performance.

Inputs:

Study hours = x_1

Sleep = x_2

Practice = x_3

Weights

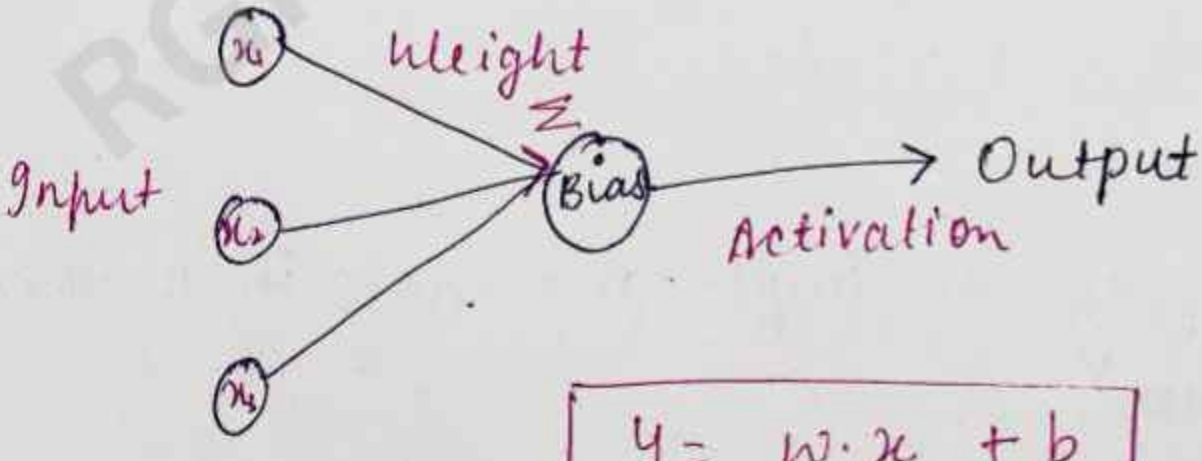
$w_1 = 3$

$w_2 = 1$

$w_3 = 2$

$y = (3 \times x_1) + (1 \times x_2) + (2 \times x_3)$

Study has heighest importance (weight = 3)



$$y = w \cdot x + b$$

RGPV ALL CONTENT

Bias

A bias is an additional parameter in a neural network that is added to the weighted sum of inputs to shift the output.

In simple words → Bias is a value that helps the model adjust the output independently of input.

Mathematical Representation -

For a neuron.

$$y = w \cdot x + b$$

for multiple inputs.

$$y = w_1 x_1 + w_2 x_2 + w_3 x_3 + \dots + w_n x_n + b$$

where b = bias.

Role of Bias

Bias play an important role in neural networks.

① shift the output

Bias move the output up and down.

REPLY ALL CONTENT

② Increases Flexibility. Helps model to fit data better.

③ Work without Input
Even if all inputs are zero, output can still be non-zero.

Real life example.

Suppose predicting salary:.

Inputs:.

Experience = x

weight = 5

Bias = 10 (base salary

$$y = 5x + 10$$

→ Even if experience is 0. salary = 10.

→ Bias = fixed starting value.

Bias in neural network.

- Every neuron has its own bias
- Bias is added after multiplying weights.

RCPV ALL CONTENT

Flow

① Multiply inputs with weights
② Add Bias
③ Apply activation function.

* Activation function → Applied to the output of a neuron ~~should be~~ ~~activated~~ to determined the final output.

It decides whether a neuron should be activated (ON) or not.

Loss FUNCTION

A function that measures the difference between actual output and predicted output.

Loss function tells how wrong the model's prediction is.

Why loss function is important.

Loss function is very important beause

① Measure Error - Tells how far prediction

RGPV ALL CONTENT

is from correct answer.

② Guides learning → Helps model improve by reducing error.

③ Used in optimization

- Works with Gradient Descent to update weights.

It tells the model.

- If prediction is close to actual value → Low Loss.
- If prediction is far from actual value → High Loss
- The main goal of training is to minimize the loss.

Simple Example → Suppose actual house prince = ₹ 50 lakh

Model predicted = ₹ 45 lakh.

Then error → difference between actual and predicted.

Loss function calculate this error mathematically.

Common Loss functions.

1. Mean Squared Error (MSE).

This is used in Regression.

PGP VAIL CONTENT

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- squares the error
- Large errors get more penality

2. Mean Absolute Error (MAE)

used in Regression

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \tilde{y}_i|$$

- Uses absolute difference
- Less affected by outliners.

3. Binary Cross Entropy.

Used in Binary classification (Yes/No, Spam/Not spam).

$$L = -(y\log(p) + (1-y)\log(1-p))$$

Difference Between Lose and cost

- Loss = error fore one training example
- Cost = average lose for all examples

PGFV ALL CONTENT

GRADIENT DESCENT

Gradient Descent is an optimization algorithm used in Machine learning (ML) to find the best values of parameters (weights and biases) that minimize the error of a model.

- It helps the model learn from data step by step.

Simple Idea.

→ Imagine you are standing on a mountain and want to reach the lowest point.

you cannot see the whole mountain. so you:

1. Check the slop around you.
2. Move a little downward.
3. Repeat until you reach the bottom.

This is how Gradient Descent works in ML.

Main Purpose / Role in ML

Gradient Descent is used to minimize the cost function (or Loss function).

- High cost → model predictions are bad.
- Low cost → model predictions are good.

PGFY ALL CONTENT

formula

$$\theta = \theta - \alpha \frac{\partial J(\theta)}{\partial \theta}$$

where:

- θ = parameter (weight)
- α = learning rate
- $J(\theta)$ = cost function.

★ we subtract because we want to move toward the minimum error.

Steps of Gradient Descent.

1. Initialize Random weights
2. Calculate prediction
3. Find error using cost function.
4. Compute Gradient (slop)
5. Update weights.
6. Repeat until error become minimum.

PGV ALL CONTENT

Types of Gradient Descent.

1. Batch Gradient Descent.
2. Stochastic Gradient Descent.
3. Mini-Batch Gradient Descent.

(1) Batch Gradient Descent

In this complete training dataset is used to calculate the gradient of the cost function before updating the model parameters.

In this method. the algorithm processes all training examples together as a single batch and then performs one update of weights in each integration.

Working process.

- Take the entire training dataset
- Calculate predicted output for all examples.
- Compute total error (cost)
- Calculate gradient using all data.
- Update weight once
- Repeat until convergence.

FOR ALL CONTENT

Characteristics

- Use full data set in every integration
- Produces smooth and stable convergence.
- Computationally expensive for large datasets.
- Require high memory.

Advantages

1. Accurate, gradient calculation.
2. Stable movement toward minimum
3. Easy mathematical analysis.

Disadvantages.

1. Very slow for large datasets.
2. High computations time
3. Not-suitable for real-time learning

PGDV ALL CONTENT

Example :- If a dataset contains 50,000 records. Batch Gradient Descent uses all 50,000 records before updating weights once.

2. Stochastic Gradient Descent (SGD)

:- It is a variation of gradient Descent where only one randomly selected training example is used to calculate the gradient and update the model parameters at a time.

- The word "stochastic" means random.
- Instead of waiting to process the entire dataset. SGD updates weights immediately after processing each training example.
- This makes learning faster but less stable.

Working Process.

1. Select one training examples.
2. Calculate predictions and error
3. Compute gradient
4. Update weights immediately.
5. Repeat for all examples.

RGPV ALL CONTENT

Characteristics

- Uses one data point at a time
- Fast updates
- Random fluctuations during learning.
- Less memory usage

Advantages

1. Faster training
2. Work well for huge datasets.
3. Can escape local minima
4. Suitable for online learning

Disadvantages.

1. Noisy updates
2. Convergence is less stable
3. Error graph fluctuate

RGIV AEL CONTENT

Example → If dataset has 50,000 records :-

- Weights update 50,000 times in one epoch.
- Each update uses only one record.

3. Mini - Batch Gradient Descent.

- Mini Batch Gradient Descent is a Hybrid approach that combines Batch Gradient Descent and Stochastic Gradient Descent.

- In this method, the training dataset is divided into small groups called mini-batches. and each mini - batch is used to update the weights.

- It balance the speed of SGD and the stability of Batch GD.

- This is the most commonly used optimizat-ion method in modern deep learning.

working process.

1. Diride data set into small batches
2. Select one mini - batch
3. Calculate gradient for that batch
4. Update weights
5. Repeat for remaining batch

BCRV ALL CONTENT

Characteristics.

- Uses small subset of data
  - faster computation
  - more stable than SGD
  - Efficient memory usage

Advantages

- fast and efficient.
- Stable convergence
- Suitable for GPU Processing
- Most practical method.

Disadvantages.

- Batch size selection is important.
- Improper batch size may reduce performance.

Example

→ If dataset has 50,000 records and batch size is 500:

- Data Divided into 100 mini-batches.
- Weights update 100 times in one epoch.

RAHUL CONTENT

MULTILAYER NETWORK

A Multilayer Network means a neural network that has more than one layer between input and output

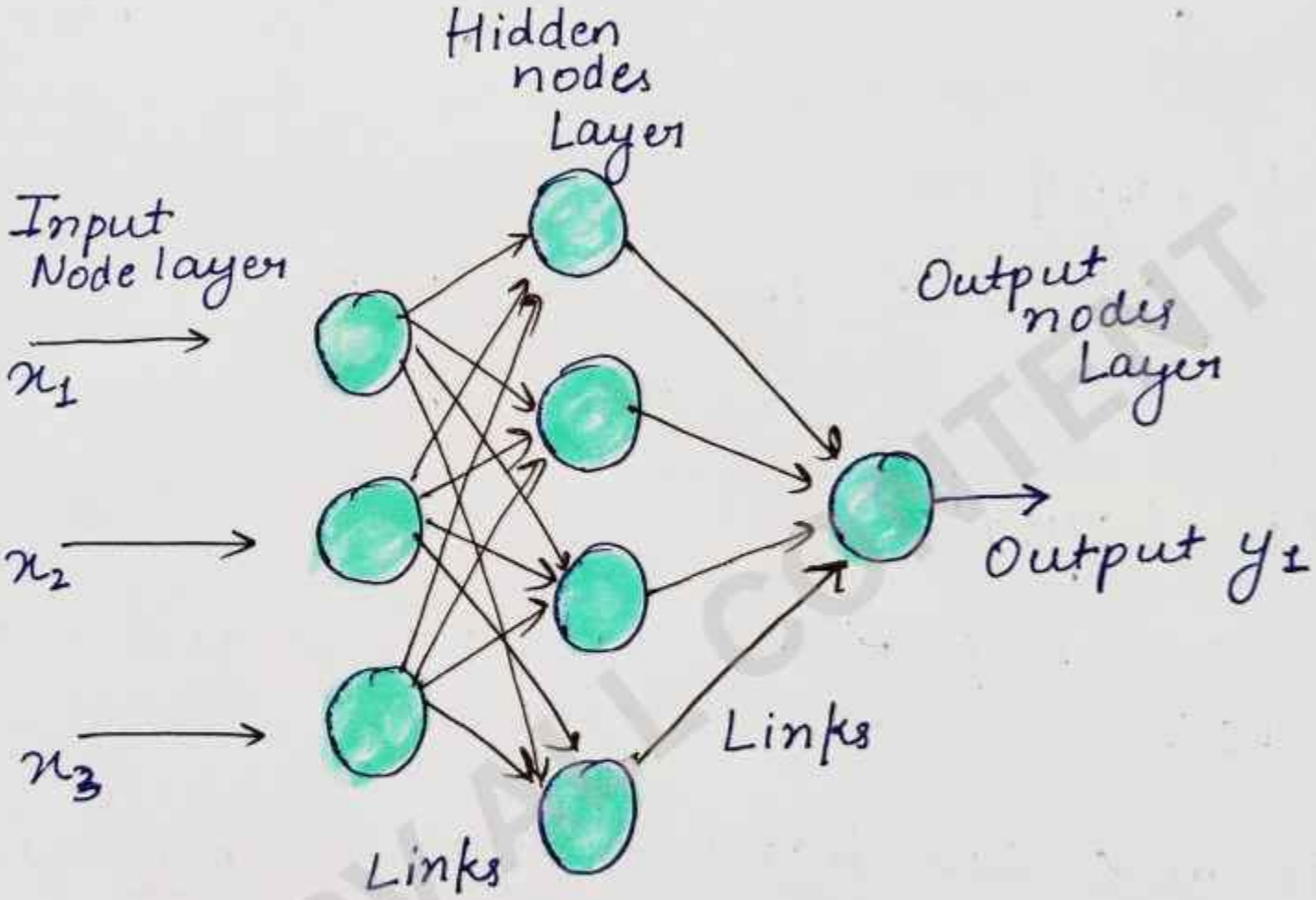
It is also called a Multilayer Neural Network or Multilayer Perceptron (MLP)

Structure of Multilayer Network

A multilayer network mainly has 3 types of layers:-

1. Input layer
   - Take input data from the user
   - Example:- image pixels, numbers, text etc.
2. Hidden Layer(s)
   - These layers perform calculations and learn patterns.
   - A network can have one or many hidden layers.
   - More hidden layers = deeper learning.
3. Output Layer:
   Gives the final result or prediction.
   Example:- . Cat or Dog classification..
   . Spam or Not Spam.

RGPV ALL CONTENT



Working of Multilayer Network.

Suppose we want to identify whenever an image is a cat or dog.

Step 1. Input Layer - The image data is given as input.

Step 2:- The Hidden layers:- detect edges, shapes, ears, eyes, patterns etc.

- Each neuron performs calculations using: weights, bias, activation function.

Step 3 Output Layer:- Finally, the network predicts: Cats or Dog.

Mathematical Idea

→ Each neuron calculates:-

$$y = f(\Sigma wx + b).$$

where:-

- x = input
- w = weight
- b = bias
- f = activation function
- y = output.

Why Multilayer Network are used.

Single-layer networks can solve only simple problems.

RGV ALL CONTENT

- Multilayer networks can solve :-
- image recognition.
- speech recognition.
- language translation.
- medical diagnosis.
- Recommendation system etc.

Advantages

1. Learns complex patterns
2. High accuracy.
3. Works well for large datasets.
4. Used in Deep learning.

Disadvantages.

1. Requires more training time.
2. Need large data
3. Computationally expensive
4. Can overfit sometimes.

Real-Life Application.

- Face Recognition
- self. driving cars
- Voice assistants
- Handwriting recognition.
- fraud detection.

FOR ALL CONTENT

BACK PROPAGATION

Back Propagation is a learning Algorithm used in neural networks to reduce errors and improve accuracy.

It helps the network learn by calculating error, sending the error backward and updating weights.

It is mainly used in Multilayer Neural Networks.

- **Back** :- error moves backward.
- **Propagation** :- spreading information.

So Back Propagation means sending the output error backward through the network to improve learning.

Steps of Back Propagation.

① **Forward Propagation** → Input is passed through the network to get output

Example:-

Input → Hidden layer → Output.

COPYRIGHT CONTENT

2. Calculate Error

~~to~~ compare : Actual Output and Predicted output.

Error formula :-

Error = Actual Output - Predicted Output.

3. Backward Propagation.

The error is sent backward from :-

- Output layer to
- Hidden layer.

The network checks :-

- which weights caused more error.

4. Update weight

Weight are adjusted to reduce error.

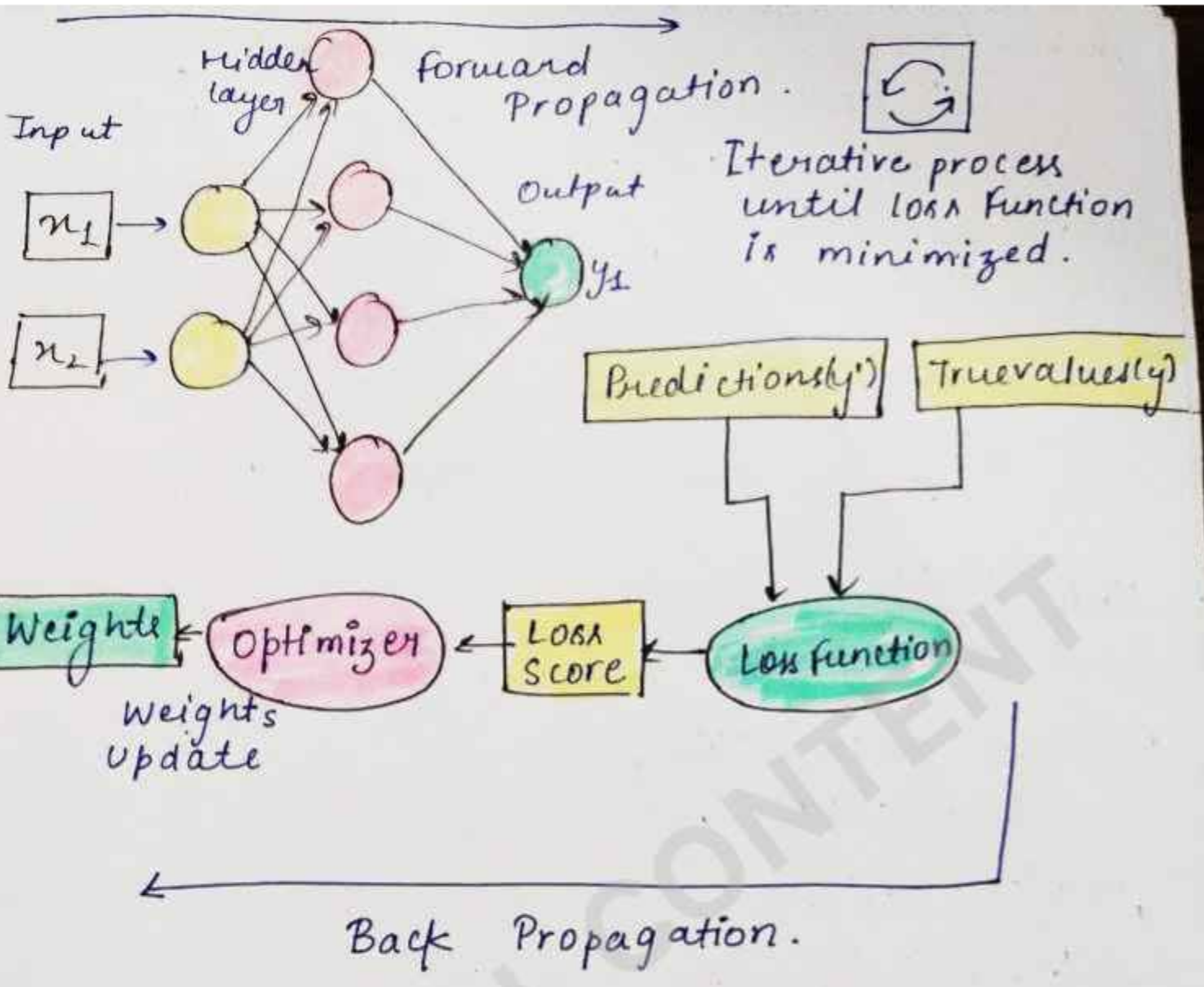
Weight update formula :-

$$W_{new} = W_{old} - \eta \frac{\partial E}{\partial W}$$

where :-

- W → weight.
- η → learning Rate.
- E → error.

SIGNAL CONTENT



Importance:-

- It helps the neural network:-
- learn from mistakes.
- Improve predictions.
- Increase accuracy.

★ Without back propagation, deep learning would not work properly.

AGRWALL CONTENT

Advantages

- Improve Accuracy
- Learn complex patterns
- Work well in deep learning.
- Reduce error automa-tically.

Disadvantages.

- Training can be slow ~~Learn complex patterns~~
- Needs large data.
- Require High computation power.

Applications.

Back Propagation is used in:-

- Image Recognition.
- Speech Recognition
- Medical dignosis
- Self-driving cars
- Chatbots

RGPV ALL CONTENT

AUTO ENCODER

- An Autoencoder is a type of neural network used to learn important features of data, Compress Data and Reconstruct the original data.
- It works automatically, so it is called Auto Encoders.

Main Idea

An autoencoder tries to:
Convert input data into a smaller form and then rebuild the original data from it.

Example

- Compressing an image.
- Removing noise from Photos.
- Features extraction.

Structure of Autoencoder.

An autoencoder has 3 main parts:-

1. Encoder
   - Compresses input data into smaller representation.
   - Extract important features.

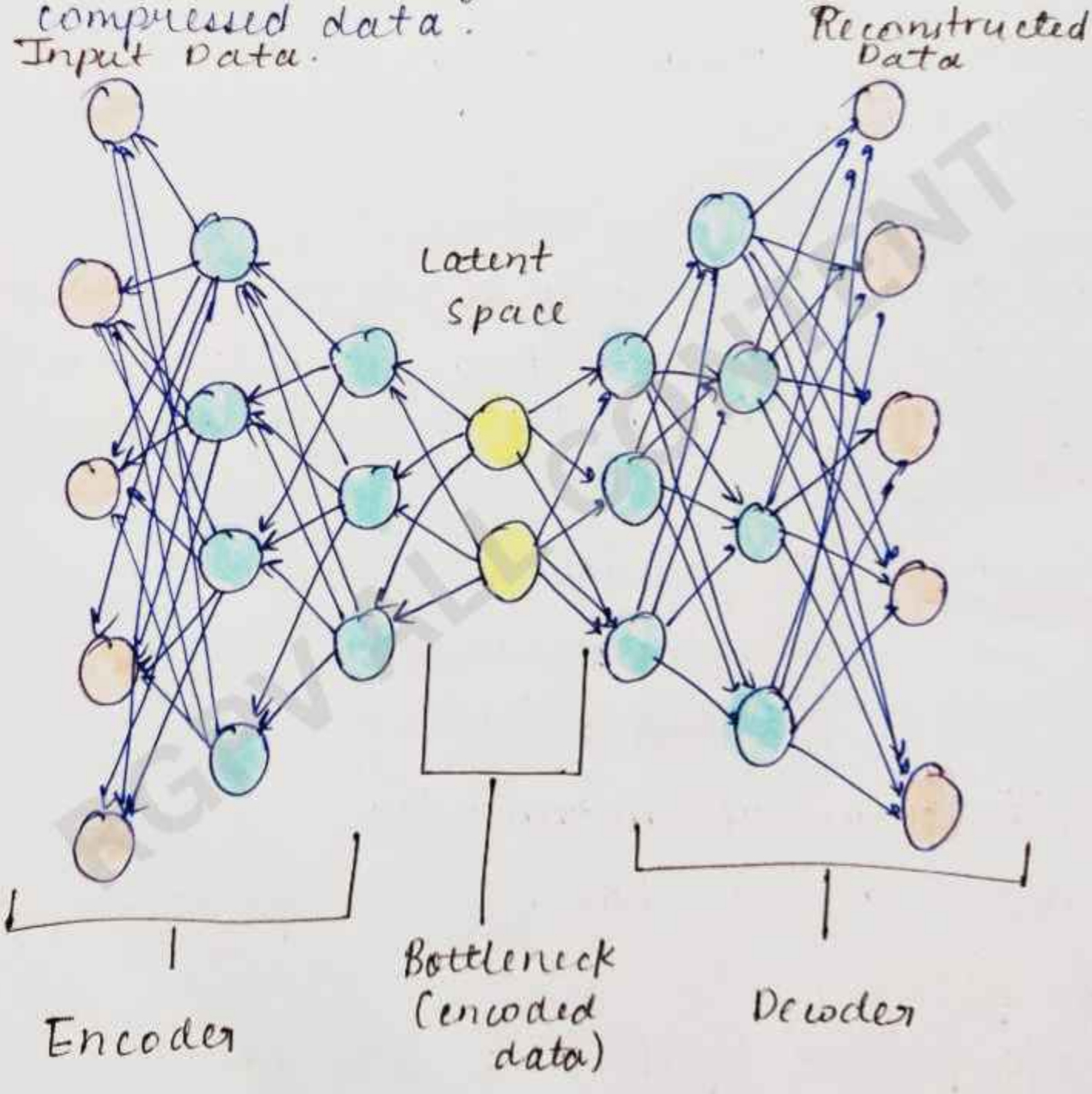
PDF FULL CONTENT

2. Bottleneck (Latent space)

- Small compressed form of data.
- Stores only important information.

3. Decoder

- Reconstructs original data from compressed data.



Working of Autoencoder.

Suppose an image is given as input.

Step 1 Encoder - The encoder compress the image.

- Large image → smaller feature representation.

Step 2 Bottleneck - Only important information is stored.

Step 3 Decoder - The Decoder rebuilds the image from compressed data.

Step 4 Compare Output - The reconstructed image is compared with original image. The network learns by reducing reconstruction error.

RGPV ALL CONTENT

Types of Auto encoders

(1) Convolution Auto encoders
(2) Sparse Auto encoders
(3) Deep Auto encoders

(1) Convolution Auto Encoders

It uses Convolutional Neural Network (CNN) layers for learning image features.

A convolutional autoencoder:

* Compresses the image using convolutional layers.
* Learns important image features.
* Reconstructs the original image.

→ It is mainly used for:-

- Image compression.
- Noise removal
- Feature extraction
- Image reconstruction

It has two main parts

(1) Encoder - The encoder:-

- uses convolution and pooling layers
- reduces image size.
- extracts important features.

RGM ALL CONTENT

Example → Image → feature Maps → Compressed Representation.

② Decoder :- The decoder :

- Reconstructs the image.
- uses deconvolution / upsampling layers.
- produces output similar to original image.

② Sparse Autoencoder :-

- In this only a few neurons are active at one time.
- It learns only important features of data
- Even if many neurons exist, the network forces most neurons to remain inactive. This creates efficient learning.

Features

- learn useful patterns.
- Reduces unnecessary learning.
- Improves feature extraction.

RGPV ALL CONTENT

(3). Deep Autoencoder

A Deep Autoencoder is an encoder with many hidden layers.

It is used for learning complex patterns in large datasets

features.

- It learns deep features
- Provide Better compression of data.
- It also Handles complex data.

Advantages of Autoencoder.

- Reduces data size
- Learns features automatically.
- Remove noise
- Useful for dimensionality reduction.

Disadvantages of Autoencoder.

- Training can be slow
- Needs large data.
- Sometimes reconstructed output is blurry.

REPLY ALL CONTENT

BATCH NORMALIZATION

Batch Normalization is a technique in ML used to make training neural networks faster, more stable, and more reliable.

Batch normalization (often called BatchNorm) normalizes the inputs of each layer so that they have.

- mean ≈ 0
- variance ≈ 1

This is done for each mini-batch during training

Need of Batch Normalization.

During data traing, the distribution of activations changes as weights updated - this is called internal covariate shift.

BatchNorm helps by :-

- stabilizing these distributions
- allowing higher learning rates
- reducing sensitivity to initialization
- acting as a mild regularizer.

RGPV ALL CONTENT

How it Works:

for each feature in a mini-batch:-

1. Compute mean

$$\mu_B = \frac{1}{m} \sum_{i=1}^{m} x_i$$

2. Compute Variance.

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^{m} (x_i - \mu_B)^2$$

3. Normalize

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

4. Scale and shift (learnable parameters):

$$y_i = \gamma \hat{x}_i + \beta$$

- γ (gamma) = Scale
- β (beta) = shift
- ϵ (epsilon) = small constant for numerical stability.

Where it is used.

BatchNorm is commonly applied:

- after linear/ dense layers.
- after convolution layers (in CNNs)
- before activation functions (most common practice)

PRIVATE CONTENT

Training Vs Inference.

- Training - uses batch statistics (mean & variance of current mini-batch)
- Inference - uses running averages collected during training

Benefits / Advantages.

- Faster convergence
- Most stable gradients
- Less overfitting (slight regularization effect)
- Enable deeper networks.

Limitations / Disadvantages.

- Depends on batch size (small batches - noisy estimates)
- Less effective in some architecture. (e.g. RNNs).
- Alternative exist like
  - Layer Normalization
  - Group Normalization

ROYAL CONTENT

DROPOUT

Dropout is a regularization technique. used to reduce overfitting in neural networks.

It works by randomly turning off some neurons during training.

During training, dropout randomly removes some neurons from the network temporarily.

These neurons

- do not participate in Forward propagation
- do not participate in Back propagation

→ This forces the network to learn more about features

Need of Dropout

Without dropout

- neural networks may memorize training data.
- model become overfitted
- poor performance on new idea.

Dropout helps the model generalize better.

PGP ALL CONTENT

L1 and L2 Regularization.

Regularization:- It is a technique in ML and Deep learning used to reduce overfitting

Overfitting happens when a model learns the training data too well and perform poorly on new data.

Regularization controls the complexity of the model by adding a penality to the loss function.

Need of Regularization.

→ Suppose a model has very large weights

→ Large weights make the model
- too sensitive to small changes.
- memorize training data.
- perform poorly on test data.

→ Regularization keeps weights smaller and simpler

BCA ALL CONTENT

General formula.

The new loss function becomes.

Total loss = Original Loss + Regularization Term

The regularization term penalizes large weights.

L1 Regularization.

L1 Regularization adds the sum of absolute value of weights to the loss function.

$$\text{Loss} = \text{Original Loss} + \lambda \sum_{i=1}^{n} |w_i|$$

where

w_i = weights.

λ = regularization strength

Large λ → Stronger penality

How L1 Works

L1 pushes some weights exactly to 0.

That means.

- unimportant features are removed
- only important features remain

so L1 performs automatic feature selection.

NOT FOR ALL CONTENT

Example of L1

Suppose weights are.

- [5, 0.2, 0.01, 7]

→ After L1 regularization:

- [4.5, 0, 0.6]

→ some weight become zero.

Properties of L1

1. Sparse model → many weight become zero.
2. Feature selection → Unnecessary features are removed automatically.
3. Simpler model → Model becomes easier to understand.

Advantages

- Reduces overfitting
- Removes useless features
- Good for high-dimensional data
- Creates compact models

PGFVALL CONTENT

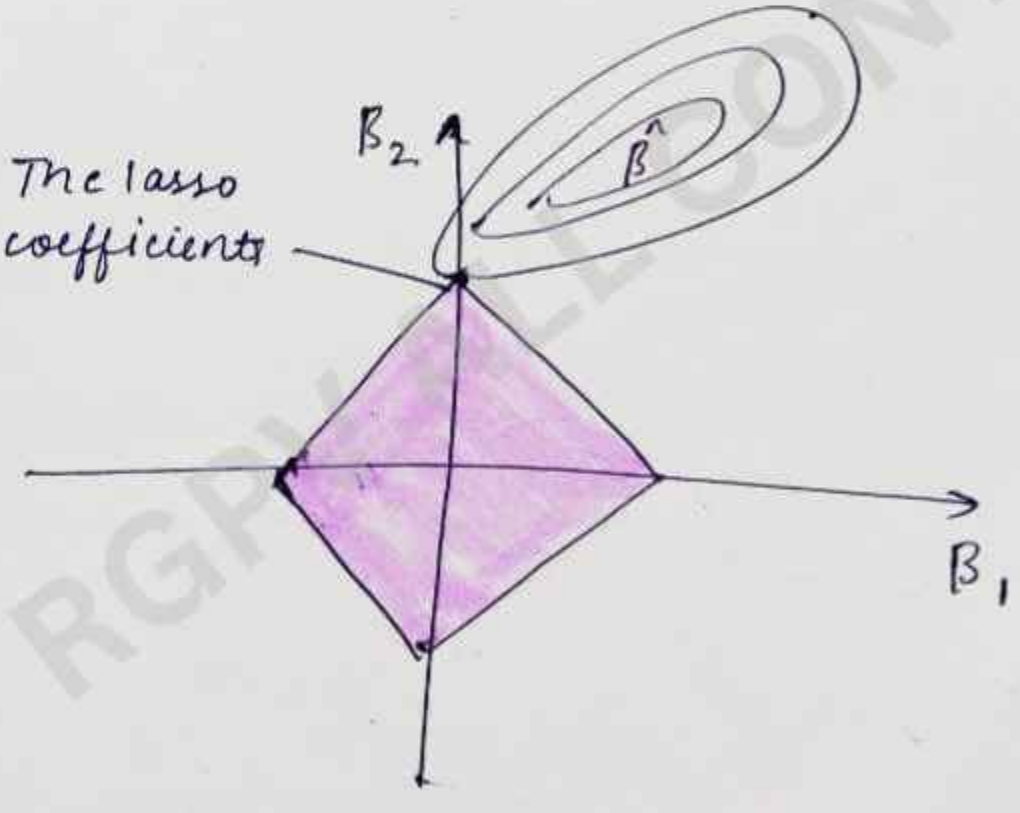
Disadvantages.

- Training may become unstable
- Problem with highly correlated features
- Can remove useful features accidentally.

Lasso Regression.

L1 regularization in linear regression is called lasso Regression.

Lasso means → Least Absolute Shrinkage and Selection operator.



LASSO

RGHT CONTENT

L2 Regularization.

L2 regularization adds the ~~square of~~ sum of squared weights to the loss function.

formula.

$$\text{Loss} = \text{Original loss} + \lambda \sum_{i=1}^{n} w_i^2$$

How L2 Works

L2 reduces weights gradually.

Weights become:-

- small
- close to zero.

But usually not exactly zero.

Example :- Original weights

[5, 0.2, 0.01, 7]

After L2:

[4.2, 0.15, 0.008, 5.8]

All weights shrink smoothly

Properties of L2

1. Weight Shrinkage → Large weights are reduced.

RGPV ALL CONTENT

2. Smooth learning - Training becomes more stable.

3. No feature Removal - Features stay in the model.

Advantages of L2:-

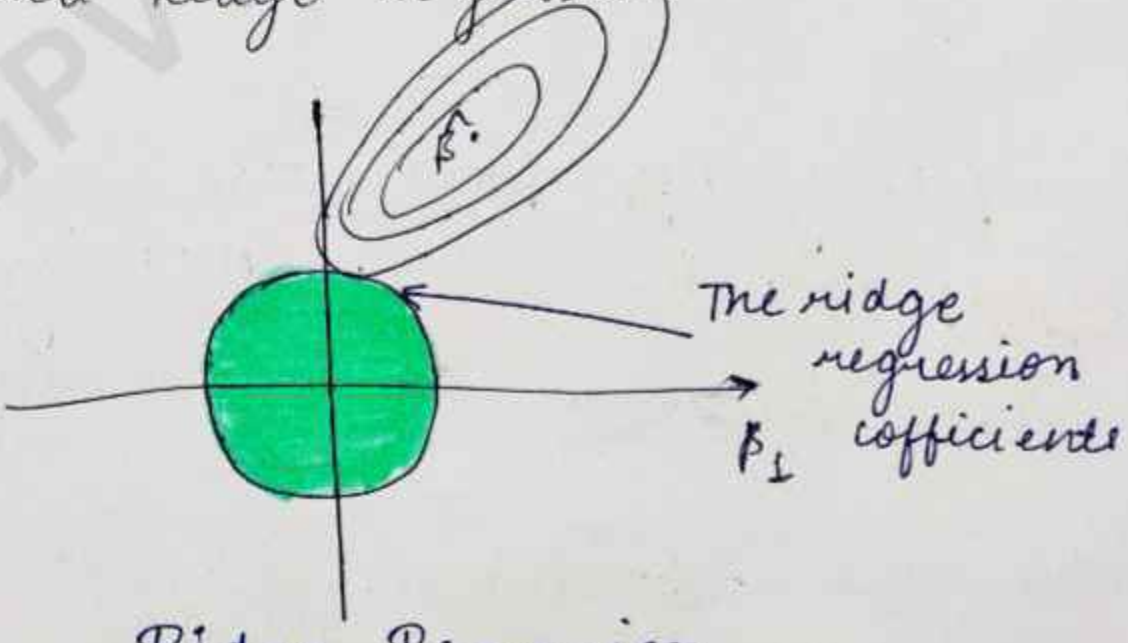
- Prevent Overfitting effectively
- Stable training
- Works well in deep learning
- Handles correlated features better

Disadvantages of L2:-

- Does not perform features selection.
- Keeps all features in model.

Ridge Regression

L2 regularization in linear regression is called Ridge Regression.



Ridge Regression

RGPV FULL CONTENT

Hyperparameter Tuning

Hyperparameter tuning is a process in ML used to find the best values of hyperparam -eters so that a model gives maximum accuracy and performance.

Hyperparameters are settings that are chosen before training the model.

They control:

- learning process
- Model complexity
- training behavior

They are not learned automatically from data.

Examples of Hyperparameters.

Hyperparameters	Meaning
1 Learning Rate →	Controls step size during training
2 Batch Size →	Number of samples processed together.

PGPV ALL CONTENT

3 Number of Epochs → How many time data passes through model.

4 Number of Hidden Layers → Depth of neural network

5 Number of Neurons → Units in each layer

6 Regularization Parameter (λ) → control overfitting

Importance

→ Correct Hyperparameters help to :-

- Improve accuracy
- Reduce overfitting
- Speed up training
- Increase model stability.

→ Bad Hyperparameters may cause :

- poor accuracy
- slow learning
- overfitting.

Example

Suppose learning rate is :-

- Too high :- model may overshoot minimum.
- Too low :- training becomes very slow.

So we find the best value.

PGDAI CONTENT

Types of Parameters

① Model Parameters → Learned automatically during training.

Examples :- weights & biases.

② Hyperparameters

Set manually before training

Example-learning rate, epochs, batch size.

Techniques of Hyperparameter Tuning.

Some Best techniques of Hyperparameter tuning are given below :-

① Manual search

② Grid search

③ Random search.

④ Bayesian Optimization.

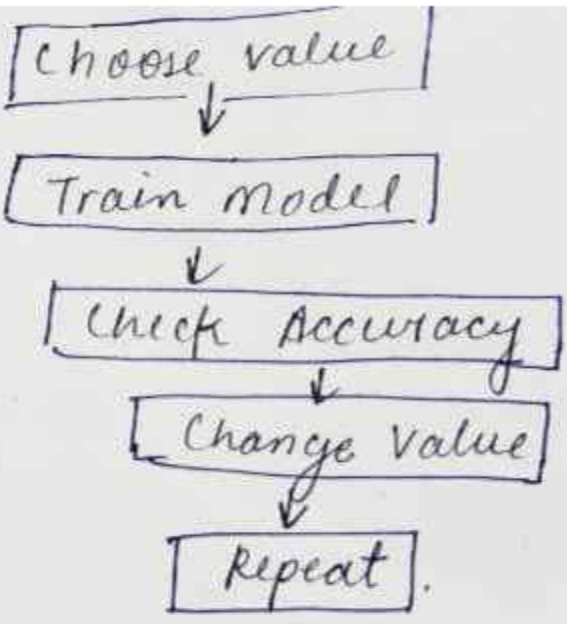
1 Manual search

In this technique, the user manually changes hyperparameters and checks performance.

Example :- learning rate 0.1
- then 0.01
- then 0.001

COPY ALL CONTENT

- It is simple and easy for beginners
- But time consuming and depend on user experience.



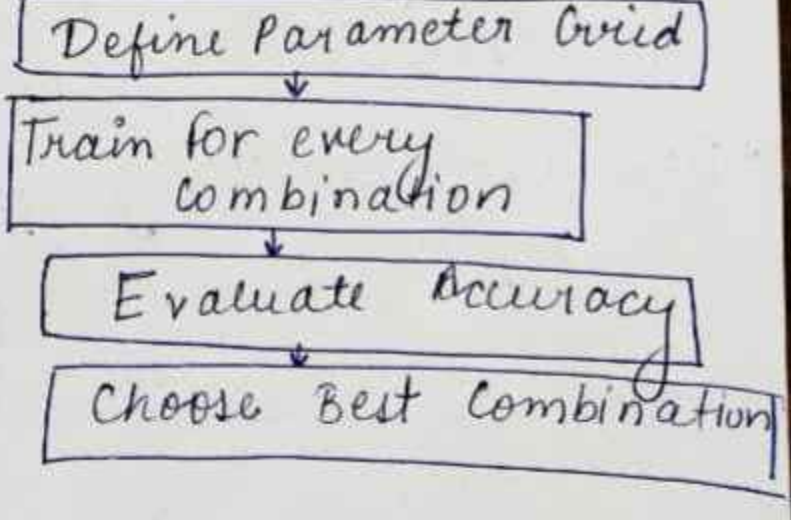
② Grid Search.

Grid search tests all possible combinations of hyperparameters.

Example →

Learning Rate	Batch size
0.1	16
0.1	32
0.01	16
0.01	32

- All combinations are checked
- It is systematic and find good combinations.
- But very slow and expensive for large datasets.



FOR ALL CONTENT

③ Random Search

- Random Search selects random combinations instead of all combinations

Example →
→ Instead of checking 100 combinations. it may test only 10 random combinations.

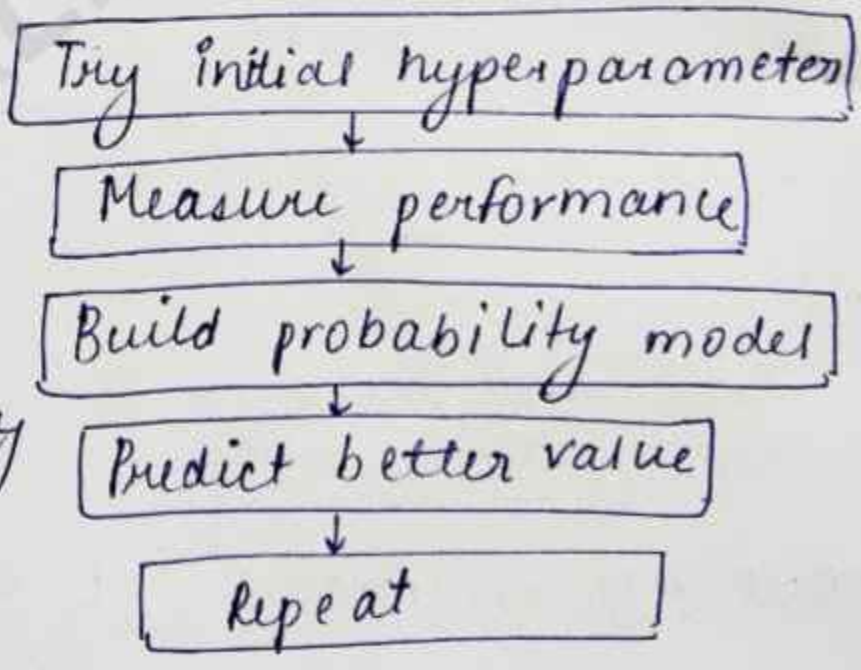
★ It is Faster than grid search and efficient in large space.

④ Bayesian Optimization.

Bayesian optimization uses probability to predict better hyperparameters.

It learns from previous trials.

- It is very efficient and need fewer experiments.
- But mathematically complex



RGM ALL CONTENT